18 Deploying Spring

This chapter covers

- Building Spring applications as either WAR or JAR files
- Building Spring applications as container images
- Deploying Spring applications in Kubernetes

Think of your favorite action movie. Now imagine going to see that movie in the theater and being taken on a thrilling audiovisual ride with high-speed chases, explosions, and battles, only to have it come to a sudden halt before the good guys take down the bad guys. Instead of seeing the movie's conflict resolved, when the theater lights come on, everyone is ushered out the door. Although the lead-up was exciting, it's the climax of the movie that's important. Without it, it's action for action's sake.

Now imagine developing applications and putting a lot of effort and creativity into solving the business problem, but then never deploying the application for others to use and enjoy. Sure, most applications we write don't involve car chases or explosions (at least I hope not), but there's a certain rush you get along the way. Not every line of code you write is destined for production, but it'd be a big letdown if none of it ever was deployed.

Up to this point, we've focused on using the features of Spring Boot that help us develop an application. There have been some exciting steps along the way, but it's all for nothing if you don't cross the finish line and deploy the application.

In this chapter, we're going to step beyond developing applications with Spring Boot and look at how to deploy those applications. Although this may seem obvious for anyone who has ever deployed a Java-based application, Spring Boot and related Spring projects have some features you can draw on that make deploying Spring Boot applications unique.

In fact, unlike most Java web applications, which are typically deployed to an application server as WAR files, Spring Boot offers several deployment options. Before we look at

how to deploy a Spring Boot application, let's consider all the options and choose a few that suit your needs best.

18.1 Weighing deployment options

You can build and run Spring Boot applications in several ways, including the following:

- Running the application directly in the IDE with either Spring Tool Suite or IntelliJ IDEA
- Running the application from the command line using the Maven `spring-boot:run` goal or Gradle `bootRun` task
- Using Maven or Gradle to produce an executable JAR file that can be run at the command line or be deployed in the cloud
- Using Maven or Gradle to produce a WAR file that can be deployed to a traditional Java application server
- Using Maven or Gradle to produce a container image that can be deployed anywhere that containers are supported, including Kubernetes environments.

Any of these choices is suitable for running the application while you're still developing it. But what about when you're ready to deploy the application into a production or other nondevelopment environment?

Although running an application from the IDE or via Maven or Gradle isn't considered a production-ready option, executable JAR files and traditional Java WAR files are certainly valid options for deploying applications to a production environment. Given the options of deploying a WAR file, a JAR file, or a container image, how do you choose? In general, the choice comes down to whether you plan to deploy your application to a traditional Java application server or a cloud platform, as described here:

- *Deploying to a Platform as a Service (PaaS) cloud*—If you're planning to deploy your application to a PaaS cloud platform such as Cloud Foundry (**https://www.cloudfoundry.org/**), then an executable JAR file is a fine choice. Even if the cloud platform supports WAR deployment, the JAR file format is much simpler than the WAR format, which is designed for application server deployment.
- *Deploying to Java application servers*—If you must deploy your application to Tomcat, WebSphere, WebLogic, or any other traditional Java application server, you really have no choice but to build your application as a WAR file.

- *Deploying to Kubernetes*—Modern cloud platforms are increasingly based on Kubernetes (**https://kubernetes.io/**). When deploying to Kubernetes, which is itself a container-orchestration system, the obvious choice is to build your application into a container image.

In this chapter, we'll focus on the following three deployment scenarios:

- Building a Spring Boot application as an executable JAR file, which can possibly be pushed to a PaaS platform
- Deploying a Spring Boot application as a WAR file to a Java application server such as Tomcat
- Packaging a Spring Boot application as a Docker container image for deployment to any platform that supports Docker deployments

To get started, let's take a look at what is perhaps the most common way of building a Spring Boot application: as an executable JAR file.

18.2 Building executable JAR files

Building a Spring application into an executable JAR file is rather straightforward. Assuming that you chose JAR packaging when initializing your project, then you should be able to produce an executable JAR file with the following Maven command:

```
$ mvnw package
```

After a successful build, the resulting JAR file will be placed into the `target` directory with a name and version based on the `<artifactId>` and `<version>` entries in the project's pom.xml file (e.g., tacocloud-0.0.19-SNAPSHOT.jar).

Or, if you're using Gradle, then this will do the trick:

```
$ gradlew build
```

For Gradle builds, the resulting JAR will be found in the build/libs directory. The name of the JAR file will be based on the `rootProject.name` property in the settings .gradle file along with the `version` property in build.gradle.

Once you have the executable JAR file, you can run it with `java -jar` like this:

```
$ java -jar tacocloud-0.0.19-SNAPSHOT.jar
```

The application will run and, assuming it is a web application, start up an embedded server (Netty or Tomcat, depending on whether or not the project is a reactive web project) and start listening for requests on the configured `server.port` (8080 by default).

That's great for running the application locally. But how can you deploy an executable JAR file?

That really depends on where you'll be deploying the application. But if you are deploying to a Cloud Foundry foundation, you can push the JAR file using the `cf` command-line tool as follows:

```
$ cf push tacocloud -p target/tacocloud-0.0.19-SNAPSHOT.jar
```

The first argument to `cf push` is the name given to the application in Cloud Foundry. This name is used to reference the application in Cloud Foundry and the `cf` CLI, as well as used as a subdomain at which the application is hosted. For example, if the application domain for your Cloud Foundry foundation is cf.myorg.com, then the Taco Cloud application will be available at **https://tacocloud.cf.myorg.com**.

Another way to deploy executable JAR files is to package them in a Docker container and run them in Docker or Kubernetes. Let's see how to do that next.

18.3 Building container images

Docker (**https://www.docker.com/**) has become the de facto standard for distributing applications of all kinds for deployment in the cloud. Many different cloud environments, including AWS, Microsoft Azure, and Google Cloud Platform (to name a few) accept Docker containers for deploying applications.

The idea of containerized applications, such as those created with Docker, draws analogies from real-world intermodal containers that are used to ship items all over the world. Intermodal containers all have a standard size and format, regardless of their contents. Because of that, intermodal containers are easily stacked on ships, carried on trains, or pulled by trucks. In a similar way, containerized applications share a common container format that can be deployed and run anywhere, regardless of the application inside.

The most basic way to create an image from your Spring Boot application is to use the `docker build` command and a Dockerfile that copies the executable JAR file from the project build into the container image. The following extremely simple Dockerfile does exactly that:

```
FROM openjdk:11.0.12-jre
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

The Dockerfile describes how the container image will be created. Because it's so brief, let's examine this Dockerfile line by line:

- *Line 1*—Declares that the image we create will be based on a predefined container image that provides (among other things) the Open JDK 11 Java runtime.
- *Line 2*—Creates a variable that references all JAR files in the project's target/ directory. For most Maven builds, there should be only one JAR file in there. By using a wildcard, however, we decouple the Dockerfile definition from the JAR file's name and version. The path to the JAR file assumes that the Dockerfile is in the root of the Maven project.
- *Line 3*—Copies the JAR file from the project's target/ directory into the container image with a generic name of app.jar.
- *Line 4*—Defines an entry point—that is, defines a command to run when a container created from this image starts—to run the JAR file with `java` `-jar` `/app.jar`.

With this Dockerfile in hand, you can create the image using the Docker command-line tool like this:

```
$ docker build . -t habuma/tacocloud:0.0.19-SNAPSHOT
```

The `.` in this command references the relative path to the location of the Dockerfile. If you are running `docker build` from a different path, replace the `.` with the path to the Dockerfile (without the filename). For example, if you are running `docker build` from the parent of the project, you will use `docker build` like this:

```
$ docker build tacocloud -t habuma/tacocloud:0.0.19-SNAPSHOT
```

The value given after the `-t` argument is the image tag, which is made up of a name and version. In this case, the image name is habuma/tacocloud and the version is 0.0.19-SNAPSHOT. If you'd like to try it out, you can use `docker run` to run this newly created image:

```
$ docker run -p8080:8080 habuma/tacocloud:0.0.19-SNAPSHOT
```

The `-p8080:8080` forwards requests to port 8080 on the host machine (e.g., your machine where you're running Docker) to the container's port 8080 (where Tomcat or Netty is listening for requests).

While building a Docker image this way is easy enough if you already have an executable JAR file handy, it's not the easiest way to create an image from a Spring Boot application. Beginning with Spring Boot 2.3.0, you can build container images without adding any special dependencies or configuration files, or editing your project in any way. That's because the Spring Boot build plugins for both Maven and Gradle support the building of container images directly. To build your Maven-built Spring project into a container image, you use the `build-image` goal from the Spring Boot Maven plugin like this:

```
$ mvnw spring-boot:build-image
```

Likewise, a Gradle-built project can be built into a container image like this:

```
$ gradlew bootBuildImage
```

This builds an image with a default tag based on the `<artifactId>` and `<version>` properties in the pom.xml file. For the Taco Cloud application, this will be something like library/tacocloud:0.0.19-SNAPSHOT. We'll see in a moment how to specify a custom image tag.

Spring Boot's build plugins rely on Docker to create images. Therefore, you'll need to have the Docker runtime installed on the machine building the image. But once the image has been created, you can run it like this:

```
$ docker run -p8080:8080 library/tacocloud:0.0.19-SNAPSHOT
```

This runs the image and exposes the image's port 8080 (which the embedded Tomcat or Netty server is listening on) to the host machine's port 8080.

The default format of the tag is docker.io/library/${project.artifactId}:${project.version}, which explains why the tag began with "library." That's fine if you'll only ever be running the image locally. But you'll most likely want to push the image to an image registry such as DockerHub and will need the image to be built with a tag that references your image repository's name.

For example, suppose that your organization's repository name in DockerHub is taco-cloud. In that case, you'll want the image name to be tacocloud/tacocloud:0.0.19-SNAPSHOT, effectively replacing the "library" default prefix with "tacocloud." To make that happen, you just need to specify a build property when building the image. For Maven, you'll specify the image name using the `spring-boot.build-image.imageName` JVM system property like this:

```
$ mvnw spring-boot:build-image \
    -Dspring-boot.build-image.imageName=tacocloud/tacocloud:0.0.19-SNAPSHOT
```

For a Gradle-built project, it's slightly simpler. You specify the image name using an `--imageName` parameter like this:

```
$ gradlew bootBuildImage --imageName=tacocloud/tacocloud:0.0.19-SNAPSHOT
```

Either of these ways of specifying the image name requires you to remember to do them when building the image and requires that you not make a mistake. To make things even easier, you can specify the image name as part of the build itself.

In Maven, you specify the image name as a configuration entry in the Spring Boot Maven Plugin. For example, the following snippet from the project's pom.xml file shows how to specify the image name as a `<configuration>` block:

```
<plugin>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-maven-plugin</artifactId>
 <configuration>
  <image>
   <name>tacocloud/${project.artifactId}:${project.version}</name>
  </image>
```

```
 </configuration>
</plugin>
```

Notice, that rather than hardcoding the artifact ID and version, we can leverage build variables to make those values reference what is already specified elsewhere in the build. This removes any need to manually bump the version number in the image name as a project evolves. For a Gradle-built project, the following entry in build.gradle achieves the same effect:

```
bootBuildImage {
 imageName = "habuma/${rootProject.name}:${version}"
}
```

With this configuration in place in the project build specification, you can build the image at the command line without specifying the image name, as we did earlier. At this point, you can run the image with `docker run` as before (referencing the image by its new name) or you can use `docker push` to push the image to an image registry such as DockerHub, as shown here:

```
$ docker push habuma/tacocloud:0.0.19-SNAPSHOT
```

Once the image is in an image registry, it can be pulled and run from any environment that has access to that registry. An increasingly common place to run images is in Kubernetes. Let's take a look at how to run an image in Kubernetes.

3.3.1 Deploying to Kubernetes

Kubernetes is an amazing container-orchestration platform that runs images, handles scaling containers up and down as necessary, and reconciles broken containers for increased robustness, among many other things.

Kubernetes is a powerful platform on which to deploy applications—so powerful, in fact, that there's no way we'll be able to cover it in detail in this chapter. Instead, we'll focus solely on the tasks required to deploy a Spring Boot application, built into a container image, into a Kubernetes cluster. For a more detailed understanding of Kubernetes, check out *Kubernetes in Action, 2nd Edition*, by Marko Lukša.

Kubernetes has earned a reputation of being difficult to use (perhaps unfairly), but deploying a Spring application that has been built as a container image in Kubernetes is re-

ally easy and is worth the effort given all of the benefits afforded by Kubernetes.

You'll need a Kubernetes environment into which to deploy your application. Several options are available, including Amazon's AWS EKS and the Google Kubernetes Engine (GKE). For experimentation locally, you can also run Kubernetes clusters using a variety of Kubernetes implementations such as MiniKube (**https://minikube.sigs.k8s.io/docs/**), MicroK8s (**https://microk8s.io/**), and my personal favorite, Kind (**https://kind.sigs.k8s.io/**).

The first thing you'll need to do is create a deployment manifest. The deployment manifest is a YAML file that describes how an image should be deployed. As a simple example, consider the following deployment manifest that deploys the Taco Cloud image created earlier in a Kubernetes cluster:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taco-cloud-deploy
  labels:
    app: taco-cloud
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taco-cloud
  template:
    metadata:
      labels:
        app: taco-cloud
    spec:
      containers:
      - name: taco-cloud-container
        image: tacocloud/tacocloud:latest
```

This manifest can be named anything you like. But for the sake of discussion, let's assume you named it deploy.yaml and placed it in a directory named k8s at the root of the project.

Without diving into the details of how a Kubernetes deployment specification works, the key things to notice here are that our deployment is named taco-cloud-deploy and (near the bottom) is set to deploy and start a container based on the image whose name is tacocloud/tacocloud:latest. By giving "latest" as the version rather than "0.0.19-

SNAPSHOT," we can know that the very latest image pushed to the container registry will be used.

Another thing to notice is that the `replicas` property is set to `3`. This tells the Kubernetes runtime that there should be three instances of the container running. If, for any reason, one of those three instances fails, then Kubernetes will automatically reconcile the problem by starting a new instance in its place. To apply the deployment, you can use the `kubectl` command-line tool like this:

```
$ kubectl apply -f deploy.yaml
```

After a moment or so, you should be able to use `kubectl get all` to see the deployment in action, including three *pods*, each one running a container instance. Here's a sample of what you might see:

```
$ kubectl get all
NAME                                  READY   STATUS    RESTARTS   AGE
pod/taco-cloud-deploy-555bd8fdb4-dln45   1/1     Running   0          20s
pod/taco-cloud-deploy-555bd8fdb4-n455b   1/1     Running   0          20s
pod/taco-cloud-deploy-555bd8fdb4-xp756   1/1     Running   0          20s
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/taco-cloud-deploy    3/3     3            3           20s


NAME                                            DESIRED   CURRENT   READY   AGE
replicaset.apps/taco-cloud-deploy-555bd8fdb4    3         3         3       20s
```

The first section shows three pods, one for each instance we requested in the `replicas` property. The middle section is the deployment resource itself. And the final section is a `ReplicaSet` resource, a special resource that Kubernetes uses to remember how many replicas of the application should be maintained.

If you want to try out the application, you'll need to expose a port from one of the pods on your machine. To do that, the `kubectl port-forward` command, shown next, comes in handy:

```
$ kubectl port-forward pod/taco-cloud-deploy-555bd8fdb4-dln45 8080:8080
```

In this case, I've chosen the first of the three pods listed from `kubectl get all` and asked to forward requests from the host machine's (the machine on which the Kubernetes cluster is running) port 8080 to the pod's port 8080. With that in place, you should be able to point your browser at http:/ /localhost:8080 to see the Taco Cloud application running on the specified pod.

3.3.2 Enabling graceful shutdown

We have several ways in which to make Spring applications Kubernetes friendly, but the two most essential things you'll want to do are to enable graceful shutdown as well as liveness and readiness probes.

At any time, Kubernetes may decide to shut down one or more of the pods that your application is running in. That may be because it senses a problem, or it might be because someone has explicitly requested that the pod be shut down or restarted. Whatever the reason, if the application on that pod is in the process of handling a request, it's poor form for the pod to immediately shut down, leaving the request unhandled. Doing so will result in an error response to the client and require that the client make the request again.

Instead of burdening the client with an error, you can enable graceful shutdown in your Spring application by simply setting the `server.shutdown` property to `"graceful"`. This can be done in any of the property sources discussed in chapter 6, including in application.yml like this:

```
server:
  shutdown: graceful
```

By enabling graceful shutdown, Spring will hold off on allowing the application to shut down for up to 30 seconds, allowing any in-progress requests to be handled. After all pending requests have been completed or the shutdown time-out expires, the application will be allowed to shut down.

The shutdown time-out is 30 seconds by default, but you can override that by setting the `spring.lifecycle.timeout-per-shutdown-phase` property. For example, to change the time-out to 20 seconds, you would set the property like this:

```
spring:
  lifecycle.timeout-per-shutdown-phase: 20s
```

While the shutdown is pending, the embedded server will stop accepting new requests. This allows for all in-flight requests to be drained before shutdown occurs.

Shutdown isn't the only time when the application may not be able to handle requests. During startup, for example, an application may need a moment to be prepared to handle traffic. One of the ways that a Spring application can indicate to Kubernetes that it isn't ready to handle traffic is with a readiness probe. Next up, we'll take a look at how to enable liveness and readiness probes in a Spring application.

3.3.3 Working with application liveness and readiness

As we saw in chapter 15, the Actuator's health endpoint provides a status on the health of an application. But that health is only in relation to the health of any external dependencies that the application relies on, such as a database or message broker. Even if an application is perfectly healthy with regard to its database connection, that doesn't necessarily mean that it's ready to handle requests or that it is even healthy enough to remain running in its current state.

Kubernetes supports the notion of liveness and readiness probes: indicators of an application's health that help Kubernetes determine whether traffic should be sent to the application, or if the application should be restarted to resolve some issue. Spring Boot supports liveness and readiness probes via the Actuator health endpoint as subsets of the health endpoint known as *health groups*.

Liveness is an indicator of whether an application is healthy enough to continue running without being restarted. If an application indicates that its liveness indicator is down, then the Kubernetes runtime can react to that by terminating the pod that the application is running in and starting a new one in its place.

Readiness, on the other hand, tells Kubernetes whether the application is ready to handle traffic. During startup, for instance, an application may need to perform some initialization before it can start handling requests. During this time, the application's readiness may show that it's down. During this time, the application is still alive, so Kubernetes won't restart it. But Kubernetes will honor the readiness indicator by not sending requests to the application. Once the application has completed initialization, it can set the readiness probe to indicate that it is up, and Kubernetes will be able to route traffic to it.

ENABLING LIVENESS AND READINESS PROBES

To enable liveness and readiness probes in your Spring Boot application, you must set `management.health.probes.enabled` to `true`. In an application.yml file, that will look

like this:

```
management:
  health:
    probes:
      enabled: true
```

Once the probes are enabled, a request to the Actuator health endpoint will look something like this (assuming that the application is perfectly healthy):

```
{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

On its own, the base health endpoint doesn't tell us much about the liveness or readiness of an application. But a request to /actuator/health/liveness or /actuator/health/readiness will provide the liveness and readiness state of the application. In either case, an up status will look like this:

```
{
  "status": "UP"
}
```

On the other hand, if either readiness or liveness is down, then the result will look like this:

```
{
  "status": "DOWN"
}
```

In the case of a down readiness status, Kubernetes will not direct traffic to the application. If the liveness endpoint indicates a down status, then Kubernetes will attempt to remedy the situation by deleting the pod and starting a new instance in its place.

CONFIGURING LIVENESS AND READINESS PROBES IN THE DEPLOYMENT

With the Actuator producing liveness and readiness status on these two endpoints, all we need to do now is tell Kubernetes about them in the deployment manifest. The tail end of the following deployment manifest shows the configuration necessary to let Kubernetes know how to check on liveness and readiness:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taco-cloud-deploy
  labels:
    app: taco-cloud
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taco-cloud
  template:
    metadata:
      labels:
        app: taco-cloud
    spec:
      containers:
      - name: taco-cloud-container
        image: tacocloud/tacocloud:latest
        livenessProbe:
          initialDelaySeconds: 2
          periodSeconds: 5
          httpGet:
            path: /actuator/health/liveness
            port: 8080
        readinessProbe:
          initialDelaySeconds: 2
          periodSeconds: 5
          httpGet:
            path: /actuator/health/readiness
            port: 8080
```

This tells Kubernetes, for each probe, to make a `GET` request to the given path on port 8080 to get the liveness or readiness status. As configured here, the first request should happen 2 seconds after the application pod is running and every 5 seconds thereafter.

How do the liveness and readiness statuses get set? Internally, Spring itself or some library that the application depends on can set the statuses by publishing an availability change event. But that ability isn't limited to Spring and its libraries; you can also write code in your application that publishes these events.

For example, suppose that you want to delay the readiness of your application until some initialization has taken place. Early on in the application lifecycle, perhaps in an `ApplicationRunner` or `CommandLineRunner` bean, you can publish a readiness state to refuse traffic like this:

```
@Bean
public ApplicationRunner disableLiveness(ApplicationContext context) {
 return args -> {
  AvailabilityChangeEvent.publish(context, ReadinessState.REFUSING_TRAFFIC);
 };
}
```

Here, the `ApplicationRunner` is given an instance of the Spring application context as a parameter to the `@Bean` method. This is necessary because the static `publish()` method needs it to publish the event. Once initialization is complete, the application's readiness state can be updated to accept traffic in a similar way, as shown next:

```
AvailabilityChangeEvent.publish(context, ReadinessState.ACCEPTING_TRAFFIC);
```

Liveness status can be updated in very much the same way. The key difference is that instead of publishing `ReadinessState.ACCEPTING_TRAFFIC` or `ReadinessState.REFUSING_TRAFFIC`, you'll publish `LivenessState.CORRECT` or `LivenessState.BROKEN`. For example, if in your application code you detect an unrecoverable fatal error, your application can request that it be killed and restarted by publishing `Liveness.BROKEN` like this:

```
AvailabilityChangeEvent.publish(context, LivenessState.BROKEN);
```

Shortly after this event is published, the liveness endpoint will indicate that the application is down, and Kubernetes will take action by restarting the application. This gives you very little time to publish a `LivenessState.CORRECT` event. But if you determine

that, in fact, the application is healthy after all, then you can undo the broken event by publishing a new event like this:

```
AvailabilityChangeEvent.publish(context, LivenessState.CORRECT);
```

As long as Kubernetes hasn't hit your liveness endpoint since you set the status to broken, your application can chalk this up as a close call and keep serving requests.

18.4 Building and deploying WAR files

Throughout the course of this book, as you've developed the applications that make up the Taco Cloud application, you've run them either in the IDE or from the command line as an executable JAR file. In either case, an embedded Tomcat server (or Netty, in the case of Spring WebFlux applications) has always been there to serve requests to the application.

Thanks in large part to Spring Boot autoconfiguration, you've been spared from having to create a web.xml file or servlet initializer class to declare Spring's `DispatcherServlet` for Spring MVC. But if you're going to deploy the application to a Java application server, you're going to need to build a WAR file. And, so that the application server will know how to run the application, you'll also need to include a servlet initializer in that WAR file to play the part of a web.xml file and declare `DispatcherServlet`.

As it turns out, building a Spring Boot application into a WAR file isn't all that difficult. In fact, if you chose the WAR option when creating the application through the Initializr, then there's nothing more you need to do.

The Initializr ensures that the generated project will contain a servlet initializer class, and the build file will be geared to produce a WAR file. If, however, you chose to build a JAR file from the Initializr (or if you're curious as to what the pertinent differences are), then read on.

First, you'll need a way to configure Spring's `DispatcherServlet`. Although this could be done with a web.xml file, Spring Boot makes this even easier with `SpringBootServletInitializr`. `SpringBootServletInitializer` is a special Spring Boot–aware implementation of Spring's `WebApplicationInitializer`. Aside from configuring Spring's `DispatcherServlet`, `SpringBootServletInitializer` also looks for any beans in the Spring application context that are of type `Filter`, `Servlet`, or `ServletContextInitializer` and binds them to the servlet container.

To use `SpringBootServletInitializer`, create a subclass and override the `configure()` method to specify the Spring configuration class. The next code listing shows `TacoCloudServletInitializer`, a subclass of `SpringBootServletInitializer` that you'll use for the Taco Cloud application.

Listing 18.1 Enabling Spring web applications via Java

```
package tacos;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.context.web.SpringBootServletInitializer;

public class TacoCloudServletInitializer
      extends SpringBootServletInitializer {
  @Override
  protected SpringApplicationBuilder configure(
                          SpringApplicationBuilder builder) {
    return builder.sources(TacoCloudApplication.class);
  }
}
```

As you can see, the `configure()` method is given a `SpringApplicationBuilder` as a parameter and returns it as a result. In between, it calls the `sources()` method that registers Spring configuration classes. In this case, it registers only the `TacoCloudApplication` class, which serves the dual purpose of a bootstrap class (for executable JARs) and a Spring configuration class.

Even though the application has other Spring configuration classes, it's not necessary to register them all with the `sources()` method. The `TacoCloudApplication` class, annotated with `@SpringBootApplication`, implicitly enables component scanning. Component scanning discovers and pulls in any other configuration classes that it finds.

For the most part, `SpringBootServletInitializer`'s subclass is boilerplate. It references the application's main configuration class. But aside from that, it'll be the same for every application where you'll be building a WAR file. And you'll almost never need to make any changes to it.

Now that you've written a servlet initializer class, you must make a few small changes to the project build. If you're building with Maven, the change required is as simple as ensuring that the `<packaging>` element in pom.xml is set to `war`, as shown here:

```
<packaging>war</packaging>
```

The changes required for a Gradle build are similarly straightforward. You must apply the `war` plugin in the build.gradle file as follows:

```
apply plugin: 'war'
```

Now you're ready to build the application. With Maven, you'll use the Maven wrapper script that the Initializr used to execute the package goal like so:

```
$ mvnw package
```

If the build is successful, then the WAR file can be found in the target directory. On the other hand, if you were using Gradle to build the project, you'd use the Gradle wrapper to execute the `build` task as follows:

```
$ gradlew build
```

Once the build completes, the WAR file will be in the build/libs directory. All that's left is to deploy the application. The deployment procedure varies across application servers, so consult the documentation for your application server's specific deployment procedure.

It may be interesting to note that although you've built a WAR file suitable for deployment to any Servlet 3.0 (or higher) servlet container, the WAR file can still be executed at the command line as if it were an executable JAR file as follows:

```
$ java -jar target/taco-cloud-0.0.19-SNAPSHOT.war
```

In effect, you get two deployment options out of a single deployment artifact!

18.5 The end is where we begin

Over the past several hundred pages, we've gone from a simple start—or start.spring.io, more specifically—to deploying an application in the cloud. I hope that you've had as much fun working through these pages as I've had writing them.

But while this book must come to an end, your Spring adventure is just beginning. Using what you've learned in these pages, go build something amazing with Spring. I can't wait to see what you come up with!

Summary

- Spring applications can be deployed in a number of different environments, including traditional application servers and PaaS environments like Cloud Foundry, or as Docker containers.
- Building as an executable JAR file allows a Spring Boot application to be deployed to several cloud platforms without the overhead of a WAR file.
- When building a WAR file, you should include a class that subclasses `SpringBootServletInitializr` to ensure that Spring's `DispatcherServlet` is properly configured.
- Containerizing Spring applications is as simple as using the Spring Boot build plugin's support for building images. These images can then be deployed anywhere Docker containers can be deployed, including in Kubernetes clusters.

Previous chapter
17 Monitoring Spring with JMX

Next chapter
Appendix. Bootstrapping Spring applications